Tools of the Trade #7 Demo Notes

AI-Assisted Black-Box API Testing with Postman and Agent Mode

1. Goal and background

Use Postman to test a Spring Boot checkout API against its functional specification. Write manual assertions, ask Postman Agent Mode to generate tests, and review whether the tests cover business rules, input partitions, boundaries, and invalid cases. This is black-box testing: expected behavior comes from the specification, not from reading the Java implementation.

Checkout rules

Area	Required behavior
Membership	REGULAR 0%; PREMIUM 10%; STUDENT 5%
SAVE10	10% off only when original subtotal is at least $75
FLAT20	$20 off only when original subtotal is at least $100
Selection	Apply larger eligible membership/promo discount; do not stack; maximum 25% cap
Shipping	$7.99 if discounted subtotal is below $50; otherwise free
Rounding	Half-up to two decimal places
Invalid input	HTTP 400 for invalid inputs

2. Get the application running

Clone the course GitHub repository provided by your instructor. Open the project folder in VS Code (File > Open Folder) and check that pom.xml is in the root. Select Terminal > New Terminal and run:

```
mvn clean test
mvn spring-boot:run
```

Leave the Spring Boot terminal running. Use the port printed in its startup log (the instructor rehearsal used 8080). If your application reports 8081, replace 8080 in the URLs below. Opening the API URL in a browser sends GET; this endpoint requires POST.

3. Create a Postman collection and requests

In Postman, create a collection named CS3300 TOTT7 Discount Testing. Choose New > HTTP Request, select POST, and enter http://localhost:8080/api/discount. Choose Body > raw > JSON; save each request inside the collection.

Request 01 — Premium + FLAT20 (used for manual tests)

```
{
  "subtotal": 100.00,
  "customerType": "PREMIUM",
  "promoCode": "FLAT20"
}
```

Click Send. Expected HTTP 200, discount 20, shipping 0, finalTotal 80. JSON may display 80 instead of 80.00; they represent the same numeric value.

Requests 02 and 03 — SAVE10 eligibility boundary

Request name	Body changes	Expected discount	Expected finalTotal
02 - SAVE10 - Below Threshold	subtotal 74.99; REGULAR; SAVE10	0	74.99
03 - SAVE10 - At Threshold	subtotal 75.00; REGULAR; SAVE10	7.50	67.50

Duplicate the first request and replace the JSON body for each boundary request. Save them. Do not copy the hardcoded $20/$80 assertions into these requests unchanged.

4. Create a fourth request and write manual tests

Create another POST request using the Request 01 body and name it 04 - Manual and AI Test Generation. In Scripts > Post-response (called Tests in older versions), enter:

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Correct discount", function () {
  pm.expect(pm.response.json().discount).to.eql(20);
});

pm.test("Correct final total", function () {
  pm.expect(pm.response.json().finalTotal).to.eql(80);
});
```

Send again. All three should pass. pm.test names a test; pm.response provides the actual HTTP response; pm.expect checks the actual value against an expected value derived from the specification.

5. AI Prompt 1 — Response validity and consistency

Open Agent Mode from Postman's AI/right sidebar, if available. Older tutorials call it Postbot. Ask:

```
Add Postman post-response tests for this checkout API. Verify numeric subtotal, discount,
shipping, and finalTotal fields; verify finalTotal equals subtotal - discount + shipping.
Preserve existing tests. Explain which tests validate the business rules versus response
structure.
```

In the instructor rehearsal, AI preserved the three manual tests and added five: four nonnegative numeric checks and one arithmetic-consistency check. Inspect the code before accepting it. An arithmetic-consistent response can still violate the discount rules.

Example of useful generated checks

```
pm.test("Response fields are numeric", function () {
  const d = pm.response.json();
  ["subtotal", "discount", "shipping", "finalTotal"].forEach(k => {
    pm.expect(d[k], k).to.be.a("number");
  });
});

pm.test("Total is internally consistent", function () {
  const d = pm.response.json();
  const cents = x => Math.round(x * 100);
  pm.expect(cents(d.finalTotal)).to.eql(
    cents(d.subtotal) - cents(d.discount) + cents(d.shipping)
  );
});
```

Think: a response with subtotal 100, discount 10, shipping 0, finalTotal 90 passes the numeric and consistency checks, but fails the FLAT20 requirement for the Premium/$100 request.

6. AI Prompt 2 — Specification-aware test generation

Ask Agent Mode to generate additional requests and assertions using the full business rules:

```
Generate additional Postman tests for this checkout API based on these rules:
REGULAR 0%, PREMIUM 10%, STUDENT 5%; SAVE10 10% when original subtotal >= $75; FLAT20 $20 when
original subtotal >= $100; do not stack discounts, use the larger eligible discount; shipping
$7.99 when discounted subtotal < $50, otherwise free; round half-up to 2 decimals; invalid
inputs return HTTP 400.
Use equivalence partitioning and boundary-value analysis. Include exact inputs and expected
results, promotion thresholds, shipping thresholds after discounts, and invalid inputs. Create
or suggest separate Postman requests with assertions. Preserve existing tests; do not invent
response fields; explain assumptions.
```

In the instructor rehearsal, AI generated 17 additional requests (20 total including three existing). It covered membership categories, FLAT20 boundaries, discount comparisons, shipping, and invalid inputs. The generated tests still required review.

7. Human review: identify and correct test-oracle errors

Two generated shipping scenarios incorrectly applied SAVE10 to subtotals below $75 (when instructor ran the same prompts). That promotion is ineligible for those orders, so the AI-generated expected outputs were wrong. The corrected approach uses PREMIUM membership without a promo to reach the shipping threshold.

Input	Discount	Discounted subtotal	Shipping	Final
PREMIUM, 55.55, no promo	5.56	49.99	7.99	57.98
PREMIUM, 55.56, no promo	5.56	50.00	0.00	50.00

Also check whether two tests cover essentially the same partition and whether all expected outputs are supported by the specification. An AI-generated assertion is not automatically a trustworthy test oracle.

8. Invalid-input check

Create or run a separate POST request with subtotal -0.01, customerType REGULAR, and no promoCode. Expected HTTP 400 and an error response. Do not leave the status-200 assertion enabled for this invalid-input request.

```
pm.test("Invalid subtotal rejected", function () {
  pm.response.to.have.status(400);
  pm.expect(pm.response.json()).to.have.property("error");
});
```

Connection to Project 2: Compare traditional and AI-assisted tests using evidence about meaningful partitions, boundary cases, defect detection, expected-result correctness, redundancy, and manual corrections. More tests are not necessarily better tests.

Quick troubleshooting

Issue	Check
Connection refused	Spring Boot is running; use the port from its startup log.
404 / 405	Use POST and the exact /api/discount path.
400 unexpectedly	Check JSON syntax, customerType values, promoCode, and subtotal.
Hardcoded test fails on another request	Update its expected values or save scenario-

	specific assertions.
No Agent Mode	Check sign-in/access; use an external AI assistant to generate Postman scripts as a fallback.